capital

Custom Harness Processor Development

Software Version 2015.1

February 2016

© 2016 Mentor Graphics Corporation
All rights reserved.

This document contains information that is proprietary to Mentor Graphics Corporation. The original recipient of this document may duplicate this document in w hole or in part for internal business purposes only, provided that this entire notice appears in all copies. In duplicating any part of this document, the recipient agrees to make every reasonable effort

to prevent the unauthorized use and distribution of the proprietary information.

This document is for information and instruction purposes. Mentor Graphics reserves the right to make changes in specifications and other information contained in this publication without prior notice, and the reader should, in all cases, consult Mentor Graphics to determine whether any changes have been made.

The terms and conditions governing the sale and licensing of Mentor Graphics products are set forth in written agreements between Mentor Graphics and its customers. No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Mentor Graphics whatsoever.

MENTOR GRAPHICS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS MATERIAL INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

MENTOR GRAPHICS SHALL NOT BE LIABLE FOR ANY INCIDENTAL, INDIRECT, SPECIAL, OR CONSEQUENTIAL DAMAGES WHATSOEVER (INCLUDING BUT NOT LIMITED TO LOST PROFITS) ARISING OUT OF OR RELATED TO THIS PUBLICATION OR THE INFORMATION CONTAINED IN IT, EVEN IF MENTOR GRAPHICS CORPORATION HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

RESTRICTED RIGHTS LEGEND 03/97

U.S. Government Restricted Rights. The SOFTWARE and documentation have been developed entirely at private expense and are commercial computer software provided with restricted rights. Use, duplication or disclosure by the U.S. Government or a U.S. Government subcontractor is subject to the restrictions set forth in the license agreement provided with the software pursuant to DFARS 227.7202- 3(a) or as set forth in subparagraph (c)(1) and (2) of the Commercial Computer Software - Restricted Rights clause at FAR 52.227-19, as applicable.

Contractor/manufacturer is:
Mentor Graphics Corporation
8005 S.W. Boeckman Road, Wilsonville, Oregon 97070-7777.
Telephone: 503.685.7000
Toll-Free Telephone: 800.592.2210
Website: www.mentor.com
SupportNet: www.mentor.com/supportnet
Contact Your Technical Writer: www.mentor.com/supportnet/documentation/reply_form.cfm

TRADEMARKS: The trademarks, logos and service marks ("Marks") used herein are the property of Mentor Graphics Corporation or other third parties. No one is permitted to use these Marks without the prior written consent of Mentor Graphics or the respective third-party owner. The use herein of a third- party Mark is not an attempt to indicate Mentor Graphics as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A current list of Mentor Graphics' trademarks may be viewed at: www.mentor.com/terms_conditions/trademarks.cfm.

End-User License Agreement: You can print a copy of the End-User License Agreement from: www.mentor.com/terms_conditions/enduser.cfm.

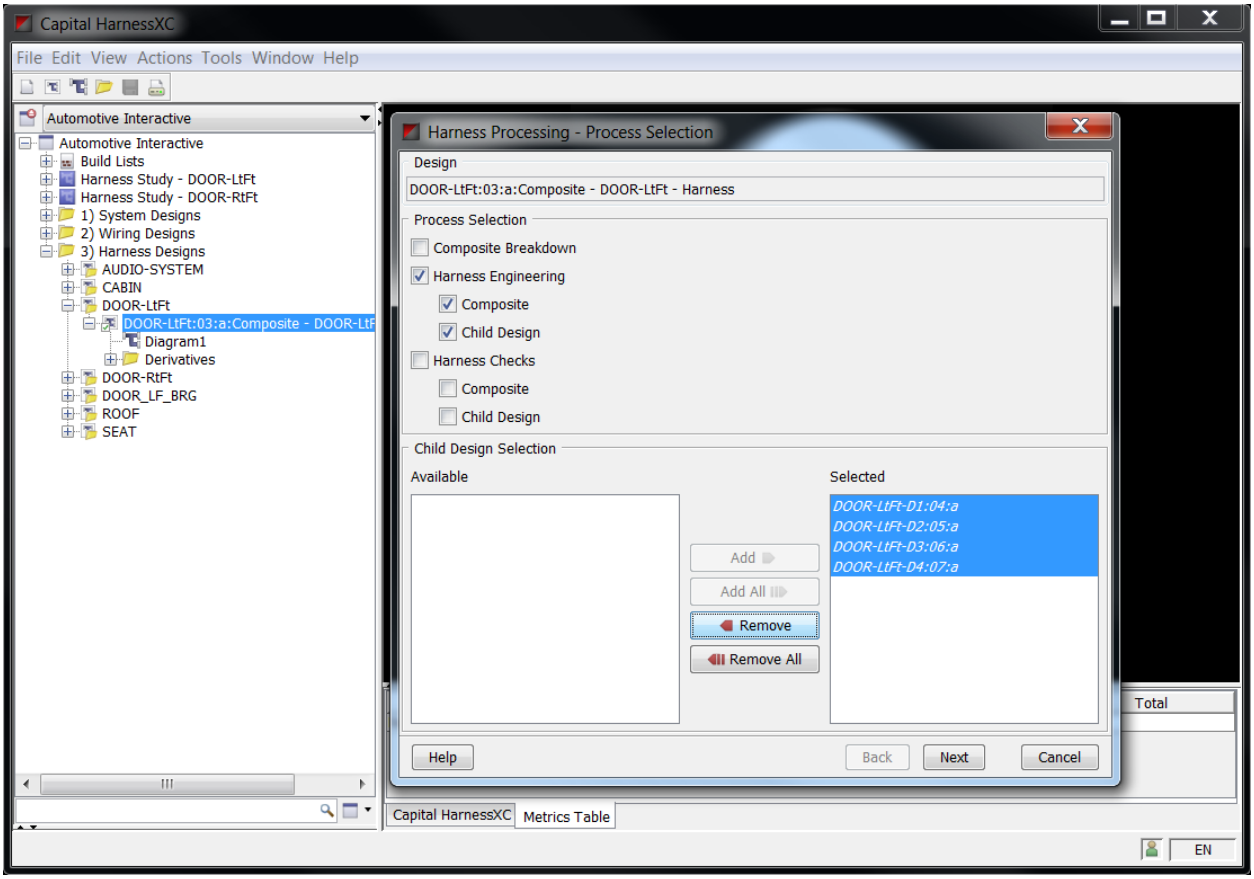
Table Of Contents

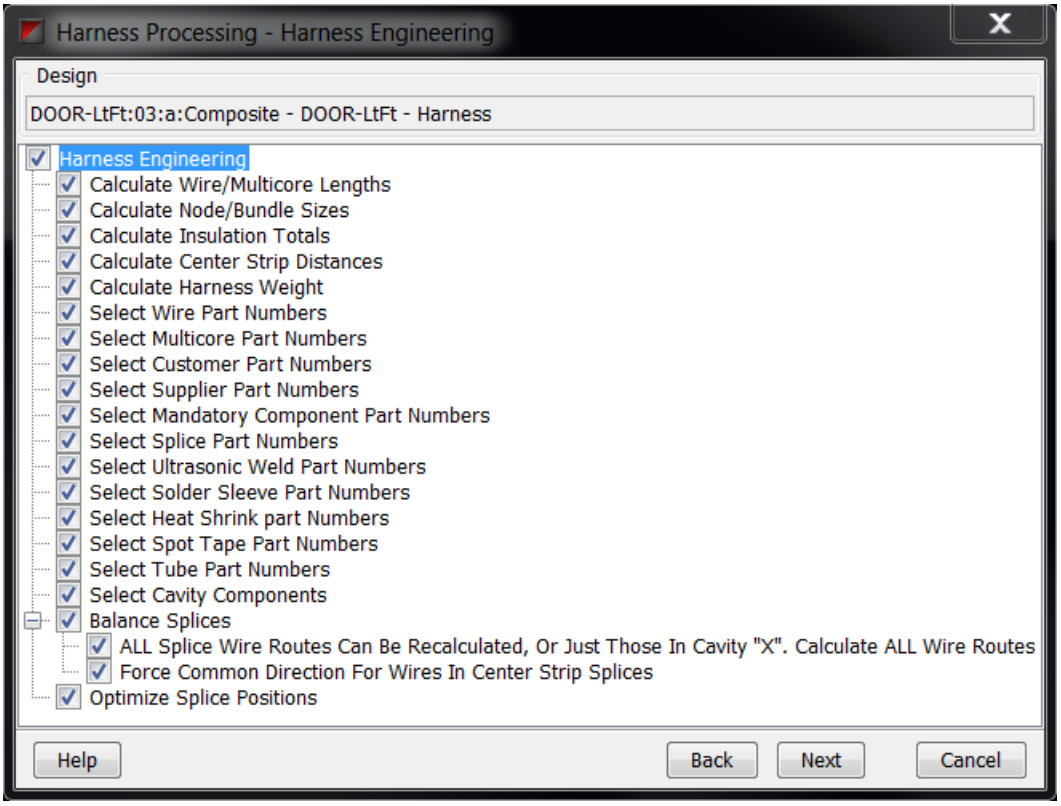
Introduction **2**
Interfaces **6**
Harness processor (any kind) 6
Output to design log 6
Custom pre-processor plugin 7
Custom post-processor plugin 7
Custom harness engineering plugin 7
Exercise **9**
Guidance **21**
Examples **22**

Introduction

This document describes the process for developing custom harness processor plugins for Capital HarnessXC or Capital ModularXC. This allows you to apply your own harness processing algorithms. If you are interested in developing custom harness processors, ensure that you have read and completed the exercises in the PluginDevelopment.pdf document before reading and completing the exercises in this document.

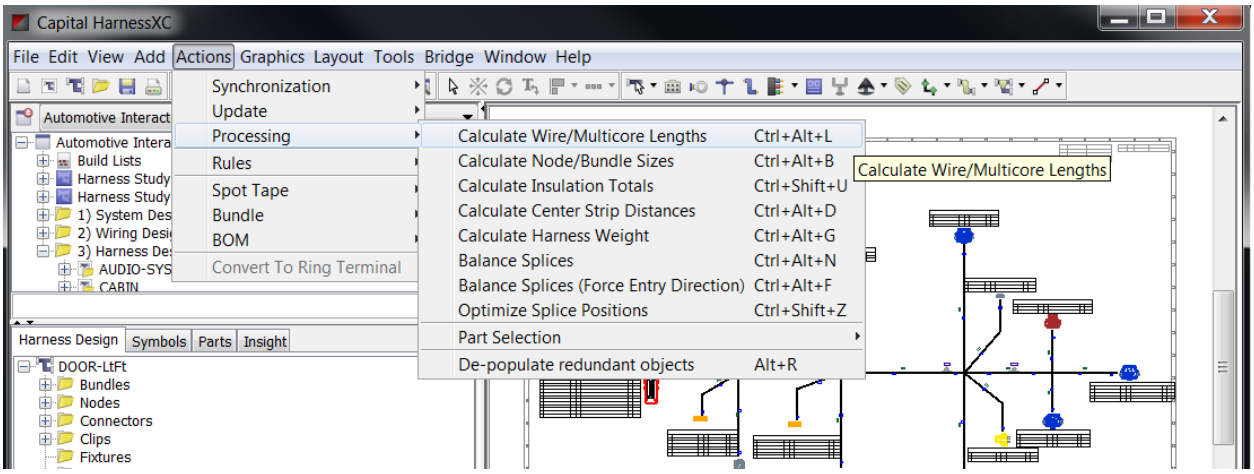
Custom harness processors are invoked from the Harness Processing Wizard in Capital HarnessXC or Capital ModularXC, whenever the plugin has been deployed in the Capital installation. Their purpose is either to supplement or replace existing Harness Engineering calculations. Therefore invocation requires the user to select Harness Engineering on the first page of the wizard, then the second page will display existing built-in calculations, as shown on the screenshots below:





Note: as Harness processing is available to run on FEM (Flexible Execution Model), custom harness processors will also be invoked this way if they are present in the Capital installation of the machine(s) which is(are) running the process.

Custom harness processors **will not be invoked** if any harness processing is run through individual actions in Capital HarnessXC or Capital ModularXC, as shown in the screenshot below:



There are 3 types of custom harness processors:

- **Pre-processor plugin** – this plugin will be executed before any processor selected in the wizard. This plugin must implement the IXPreHarnessProcessor interface.

- **Post-processor plugin** – this plugin will be executed after any processor selected in the wizard. This plugin must implement the IXPostHarnessProcessor interface.

- **Custom harness engineering plugin** – this plugin will be executed instead of the corresponding processor selected in the wizard. This plugin must implement the IXHarnessEngineering interface.

In addition, two more types are available when IXHarnessEngineering is implemented as well as either one of the following:

- IXPreHarnessProcessor – the plugin will be invoked just before the processor selected in the wizard (and will not replace it)
- IXPostHarnessProcessing – the plugin will be invoked just after the processor selected in the wizard (and will not replace it)

Each of these Java interfaces will have one `process()` method exposed by the abstract IXHarnessProcessor super-interface (which cannot be directly implemented). This method is passed in the harness design (IXHarnessDesign) which is being processed and an output object (IXHarnessProcessingOutput) which allows logging messages to the design log (available in the application in the Tools > Design Log menu item).

The implementation will navigate business objects in the design, read some of their attributes as input and set some other attributes as output, using the IXAttributeSetter interface.

Notes:

- If your implementation modifies the "Part Number" attribute of a business object, then Capital will automatically assign this part number to the object after execution of the plugin.
- It is possible to modify certain attributes from within a custom Harnesss Processor, which are read-only in any other context (other type of plugin or interactive application). Those attributes are the following:
  - "ModifiedLength"
  - "UnmodifiedLength"
  - "Weight"
  - "Width"
  - "WidthWithInsulations"

Finally, the implementation will return a boolean value which indicates a successful execution if true. If false, it indicates a failure, in which case, the following will happen:

- Attribute changes you may have requested using IXAttributeSetter will not be applied

- Consequently, part number assignation will not be performed (in case you have requested a change of the “Part Number” attribute)

In addition to the methods that are specific to custom constraints, you will need to implement the `getName()`, `getDescription()` and `getVersion()` methods that all plugins need to implement.

Interfaces

Harness processor (any kind)

This abstract interface exposes the `process()` method, common to all three types of custom harness processors.

```
public interface IXHarnessProcessor
{
     boolean process(IXHarnessDesign design,
                     IXHarnessProcessingOutput output)
}
```

Given the harness design being processed, executes the processing by setting attribute values on the business objects present in the design.

Parameters:
`design` – the harness design being processed
`output` – the object used to add messages to the design log

Returns:
`boolean` – true for successful processing, false to cancel any attribute change request

Output to design log

Harness processor implementations can log their own message to the design log by using the output object passed to their `process()` method.

```
public interface IXHarnessProcessingOutput
{
     boolean log(Severity severity, String message)
}
```

Adds the specified message to the design log.

Parameters:
`severity` – the severity of the message (one of Information, Warning or Error)
`message` – the message string

Custom pre-processor plugin

This is a concrete custom processor which is invoked before any of the other types of processors (including non-custom ones) are executed.

```
public interface IXPreHarnessProcessor extends IXHarnessProcessor
{
}
```

There is no method specific to this interface.

Custom post-processor plugin

This is a concrete custom processor which is invoked after any of the other types of processors (including non-custom ones) are executed.

```
public interface IXPostHarnessProcessor extends
IXHarnessProcessor
{
}
```

There is no method specific to this interface.

Custom harness engineering plugin

This is a concrete custom processor which is invoked instead of the non-custom harness processor it is designed to override.

```
public interface IXHarnessEngineering extends IXHarnessProcessor
{
       Type getType()
}
```

Returns:
`Type` – The type of harness processor overridden by this custom processor

If the plugin implementation also implements the `IXPreHarnessProcessor` interface, then it will be invoked immediately before the non-custom harness processor returned by the `getType()` method.
If the plugin implementation also implements the `IXPostHarnessProcessor` interface, then it will be invoked immediately after the non-custom harness processor returned by the `getType()` method.

The following is a table that shows which engineering processor corresponds to which type:

Engineering Task	IXHarnessEngineering.Type
Calculate Wire/Multicore Lengths	`WireMulticoreLength`
Calculate Node/Bundle Sizes	`BundleNodeSizes`
Calculate Insulation Totals	`InsulationTotals`
Calculate Center Strip Distances	`CentreStripDistances`
Calculate Harness Weight	`HarnessWeight`
Select Wire Part Numbers	`SelectWirePartNo`
Select Multicore Part Numbers	`SelectMulticorePartNo`
Select Customer Part Numbers	`SelectCustPartNo`
Select Supplier Part Numbers	`SelectSuppPartNo`
Select Mandatory Component Part Numbers	`SelectMandatoryPartNo`
Select Splice Part Numbers	`SelectSlicePartNo`
Select Solder Sleeve Part Numbers	`SelectSolderSleevePartNo`
Select Ultrasonic Weld Part Numbers	`SelectUltraSonicPartNo`
Select Heat Shrink part Numbers	`SelectHeatShrinkPartNo`
Select Spot Tape Part Numbers	`SelectSpotTapePartNo`
Select Tube Part Numbers	`InsulationTotals`
Select Cavity Components	*None*
Balance Splices	*None*
Optimize Splice Positions	*None*

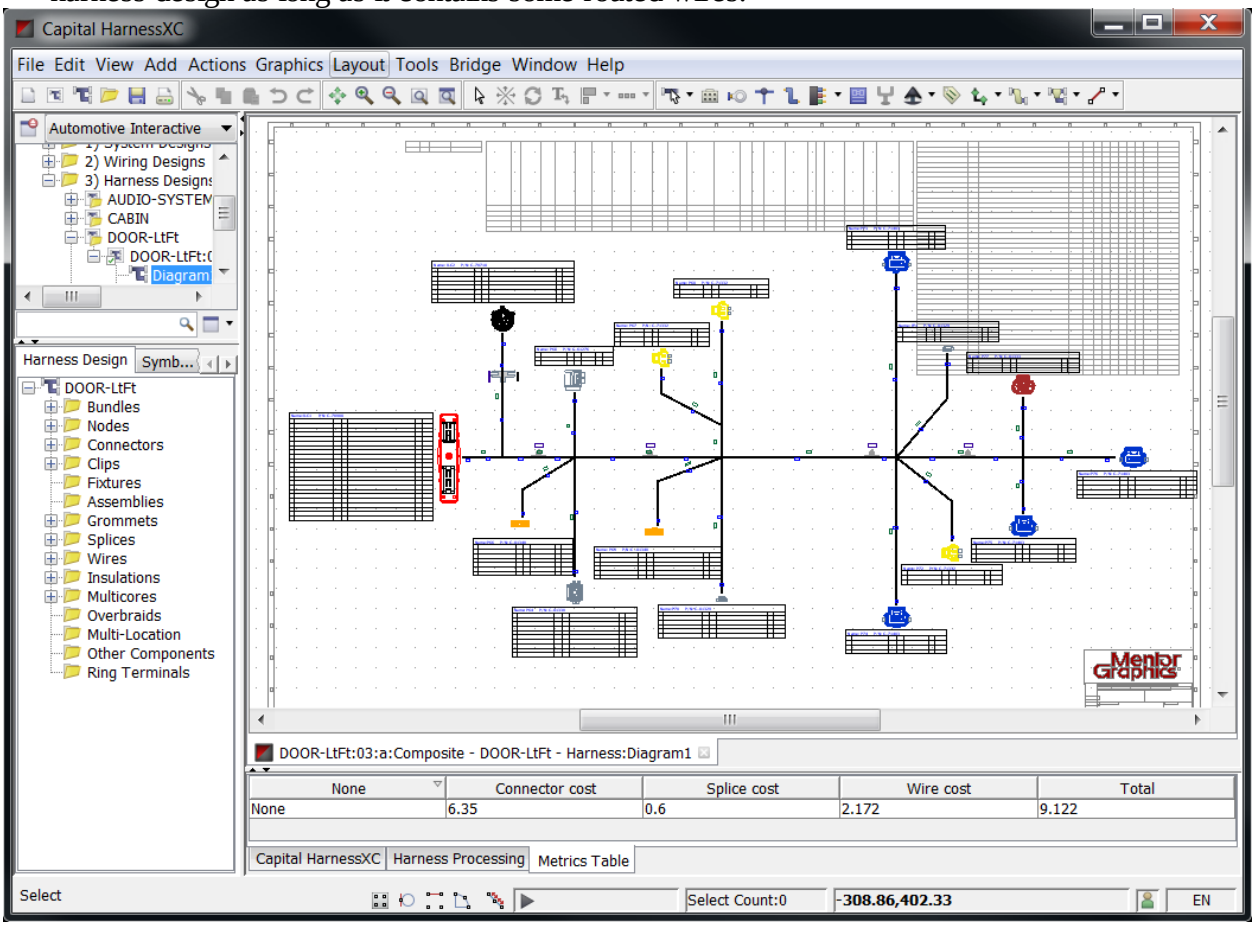
Exercise

In this exercise, you will develop a custom harness processor for calculating wire lengths, in place of the standard processing algorithm provided in the Harness Processing Wizard in Capital. The algorithm you will provide instead will, purely as an exercise, use the wire name length (number of characters) to assign to the wire length.

You need to be familiar with Capital HarnessXC to do this exercise.

1) First, you should start by setting up some test data.

   a. For this exercise, we have used the Capital starter data, but you can use your own harness design as long as it contains some routed wires.



   b. We also recommend displaying the Wire Table, which allows you to quickly check the lengths set by the plugin (right-most column), the few lengths displayed in the

screenshot below being as calculated by the Capital processor ("Calculate Wire/Multicore Lengths"):

Wire No.	Index	Option	Part No.	From	Cav.	To	Cav.	Spec	Size	Col.	M/C	Length
2N-SPKR-LF-1-2473	1	Audio-Base		ILC1	8	P65	2	0.75	0.75	R	MC2096	410
2N-SPKR-LF-2-2475	2	Audio-Base		ILC1	20	P65	1	0.75	0.75	B	MC2096	410
4N-FUEL-LID-2250	3	FuelLid && LHD		P67	2	ILC1	23	1.0	1	G		695
4N-INACTIVE-2253	4	FuelLid && LHD		P67	1	P71	1	1.0	1	G		515
4N-INACTIVE-2254	5	FuelLid && LHD && Liftgate		P68	1	P67	1	1.0	1	G		235
4N-POWER-1-2256	6	FuelLid && LHD || LHD && Liftgate		ILC1	25	P71	3	1.0	1	R		830
4N-TRUNK-2258	7	LHD && Liftgate		P68	2	ILC1	15	1.0	1	G		695
5N-CTSY-LP-2261	8	IllumEntry && LHD		SP404	X	P64	8	0.5	0.5	LG		25
5N-CTSY-LP-2263	9	IllumEntry && LHD		SP404	X	P77	A	0.5	0.5	LG		
5N-CTSY-LP-2264	10	IllumEntry && LHD		SP404	X	ILC1	6	0.5	0.5	LG		5
5N-GND-CTSY-LP-2660	11	DoorLock-Drvr && LHD || DoorLock-Pass && RHD || LH...		ILC1	12	ILC2	3	1.0	1	B		230
5N-GND-CTSY-LP-2661	12	DoorLock-Drvr && LHD		SP449	X	P76	3	1.0	1	B		15
5N-GND-CTSY-LP-2662	13	DoorLock-Drvr && IllumEntry && LHD		P77	B	P76	3	1.0	1	B		235

2) The next step is to create a new custom harness engineering plugin from a template.

   a. Ensure that your Java development environment has been set-up. If you have completed the Java plugin exercise then it should already be setup. If you have not, then you should stop this exercise and complete it now.

   b. Open a UNIX/DOS console. Ensure that the environment variables required for Java, ANT and Capital have been setup in this console. Cd into the following folder within the Capital installation:

      `doc/plugin/examples/Java/src`

   c. Create a new folder in the following location within the Capital installation folder:

      `doc/plugin/examples/Java/src/exercise`

      The folder `src` is the directory where all the example Java plugin source code has been placed. This should exist already. The new folder `exercise` is the package in which your plugin will reside. You will also find some template plugins in the following folder:

      `doc/plugin/examples/Java`

   d. Copy one of the templates, for instance `TemplatePlacementConstraint.java` into the new folder you have just created and rename the file into `ExampleWireLengthProcessor.java`

   e. Edit this new file.

   f. Ensure the plugin is in the correct package. This can be done by changing the following line at the start of the file (before the imports):

```
package exercise;
```

If you use a template as a starting point for your new plugin, you must remember to specify the package it is in.
You must also rename the class and any constructors as per the file name, and implement the harness engineering interface (which you need to add as an import statement) as follows:

```
import com.mentor.chs.plugin.harness.IXHarnessEngineering;

public class ExampleWireLengthProcessor implements
IXHarnessEngineering
{

    public ExampleWireLengthProcessor()
    {
    }
```

g. Change the name returned by the plugin () by changing the string returned by the `getName()` method:

```
public String getName()
{
    return "Name-based wire length processor";
}
```

h. Change the description returned by the plugin by changing the string returned by the `getDescription()` method:

```
public String getDescription()
{
    return "Harness engineering plugin for name-based wire
length calculation as an exercise";
}
```

i. Remove all other methods from the template except the `getVersion()` one.

j. Since your class implements the `IXHarnessEngineering` interface, you need to implement the `getType()` method and return the type corresponding to the "Calculate Wire/Multicore Lengths" processor (the available types enumeration is exposed in the `IXHarnessEngineering` interface):

```
public Type getType()
{
    return Type.WireMulticoreLength;
```

```
}
```

k. You also need to implement the process() method, but for the moment you can leave the method empty:

```
public boolean process(IXHarnessDesign design,
IXHarnessProcessingOutput output)
{
      return true;
}
```

l. You will also need to add import statements in the top section of the file, for the method argument types:

```
import com.mentor.chs.plugin.harness.IXHarnessEngineering;
import com.mentor.chs.api.IXHarnessDesign;
import
com.mentor.chs.plugin.harness.IXHarnessProcessingOutput;
```

m. Remember to save the file when these changes are completed.

n. Back at the UNIX/DOS console, ensure that you are in the following folder:

```
doc/plugin/examples/Java
```

o. Edit the `build.xml` file in order to exclude from the build any other harness processor plugins provided in your Capital installation – this will be on the first `<javac` line encountered in this file:

```
<javac srcdir="${srcdir}" destdir="${destdir}"
excludes="**/login/**, **/autolink/**, **/bridge/**,
**/LibraryDrivenScoping/**,
**/LibraryAndDesignDrivenScoping/**, **/harnessprocessing/**"
               debug="${debug}" includeantruntime="false">
```

p. Build your new plugin by typing the following command:

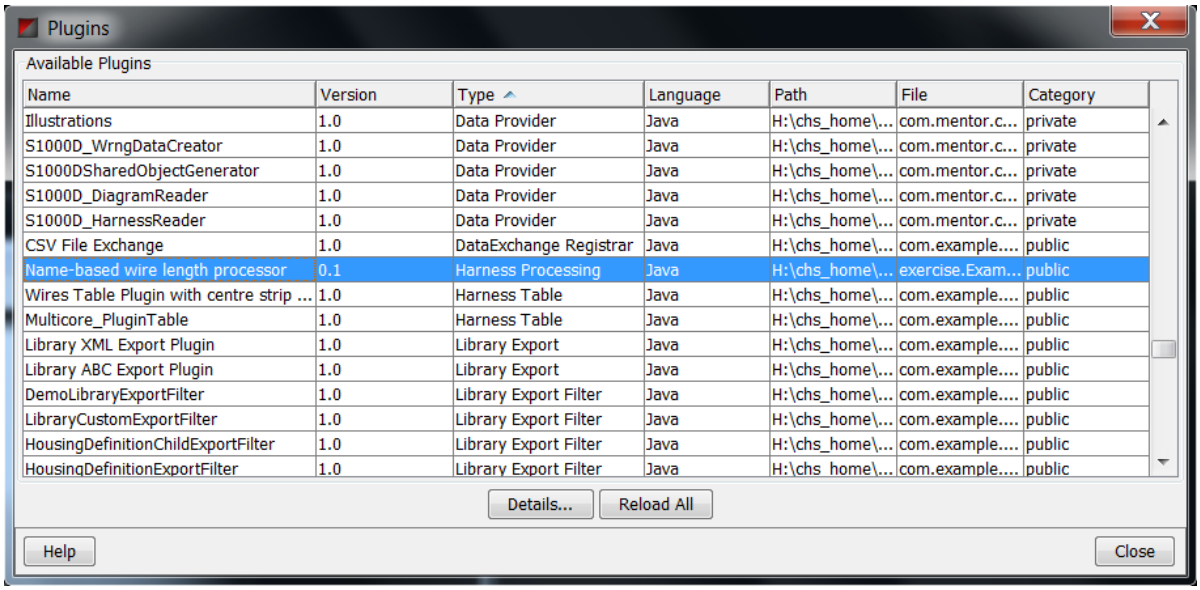
```
ant
```

q. If everything is fine, you will eventually get a "BUILD SUCCESSFUL" message. If you do not, you will need to check the environment variables or check for (and fix) errors in the `ExampleWireLengthProcessor.java` file and try again.

r. Go back to Capital HarnessXC.

s. From the main menu, click on the File > Plugins… menu

t. Press the **Reload All** button on the Plugins dialog. You will see your new plugin appear in the list with the other available plugins. See the selected row in the image below:



Name	Version	Type	Language	Path	File	Category
Illustrations	1.0	Data Provider	Java	H:\chs_home\...	com.mentor.c...	private
S1000D_WrngDataCreator	1.0	Data Provider	Java	H:\chs_home\...	com.mentor.c...	private
S1000DSharedObjectGenerator	1.0	Data Provider	Java	H:\chs_home\...	com.mentor.c...	private
S1000D_DiagramReader	1.0	Data Provider	Java	H:\chs_home\...	com.mentor.c...	private
S1000D_HarnessReader	1.0	Data Provider	Java	H:\chs_home\...	com.mentor.c...	private
CSV File Exchange	1.0	DataExchange Registrar	Java	H:\chs_home\...	com.example....	public
Name-based wire length processor	0.1	Harness Processing	Java	H:\chs_home\...	exercise.Exam...	public
Wires Table Plugin with centre strip ...	1.0	Harness Table	Java	H:\chs_home\...	com.example....	public
Multicore_PluginTable	1.0	Harness Table	Java	H:\chs_home\...	com.example....	public
Library XML Export Plugin	1.0	Library Export	Java	H:\chs_home\...	com.example....	public
Library ABC Export Plugin	1.0	Library Export	Java	H:\chs_home\...	com.example....	public
DemoLibraryExportFilter	1.0	Library Export Filter	Java	H:\chs_home\...	com.example....	public
LibraryCustomExportFilter	1.0	Library Export Filter	Java	H:\chs_home\...	com.example....	public
HousingDefinitionChildExportFilter	1.0	Library Export Filter	Java	H:\chs_home\...	com.example....	public
HousingDefinitionExportFilter	1.0	Library Export Filter	Java	H:\chs_home\...	com.example....	public

Leave this dialog open for now.

3) At this point you have a new plugin. However, it has not yet been modified to implement the required behavior. This is your next objective.

   a. Edit the `ExampleWireLengthProcessor.java` file.

   b. Edit the `process()` method in the file and add the following code:

```
public boolean process(
     IXHarnessDesign design,
     IXHarnessProcessingOutput output)
{
      for (IXWire wire : design.getConnectivity().getWires())
{
                // As an exercise use an arbitrary value for the
length such as the name length
                String lengthValue =
Integer.toString(wire.getAttribute("Name").length());
                IXAttributeSetter setter =
wire.getAttributeSetter();
                setter.addAttribute(IXAttributes.ModifiedLength,
lengthValue);

                // Add an information message to the output
                output.log(IXHarnessProcessingOutput.
Severity.Information,
                wire.getAttribute(IXAttributes.Name) + " - " +
lengthValue);
```

```
        }

        return true;
}
```

Note the use of the `getAttribute()` method call to get the name of the slot and the device, and the use of the `IXAttributeSetter` interface to request an attribute change.
Also don't forget to add import statement for the new types used:

```
import com.mentor.chs.api.IXWire;
import com.mentor.chs.plugin.IXAttributeSetter;
import com.mentor.chs.api.IXAttributes;
```

c. Change the version number of the plugin, by editing the getVersion() method:

```
public String getVersion()
{
        return "0.2";
}
```

d. Save the file.

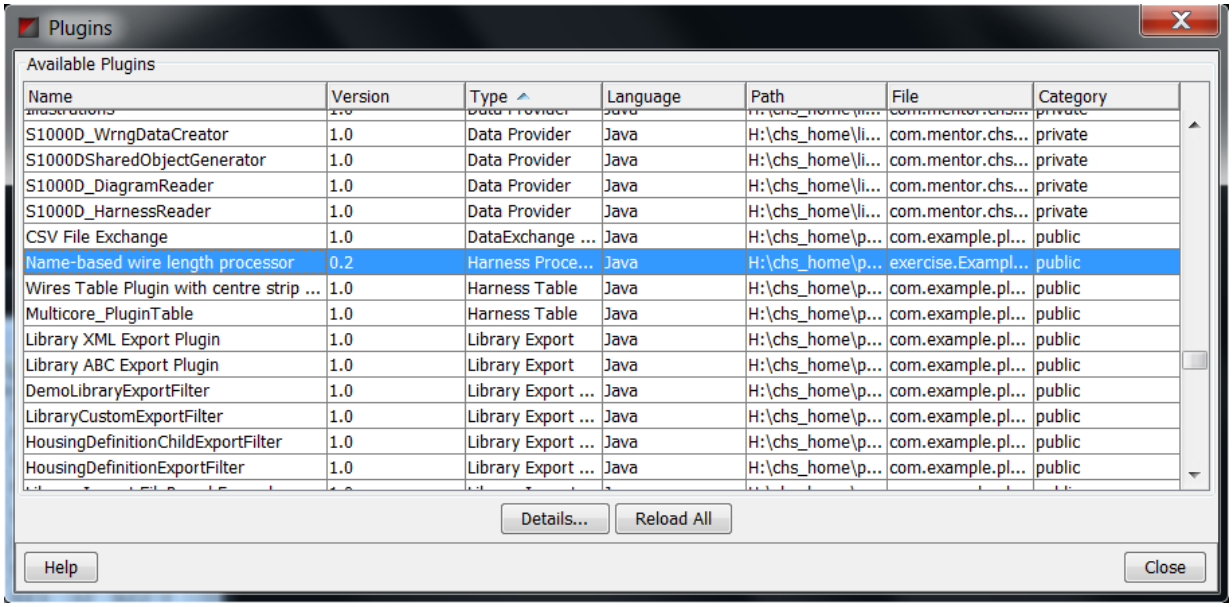
e. Rebuild your plugin by typing the following command in the DOS/UNIX console:

```
ant
```

f. If everything is fine, you will eventually get a "BUILD SUCCESSFUL" message. If you do not, you will need to check for (and fix) errors in the `ExampleWireLengthProcessor.java` file and try again.

g. Go back to Capital HarnessXC.

h. Press the **Reload All** button on the Plugins dialog. You will see the version number of your new plugin change:



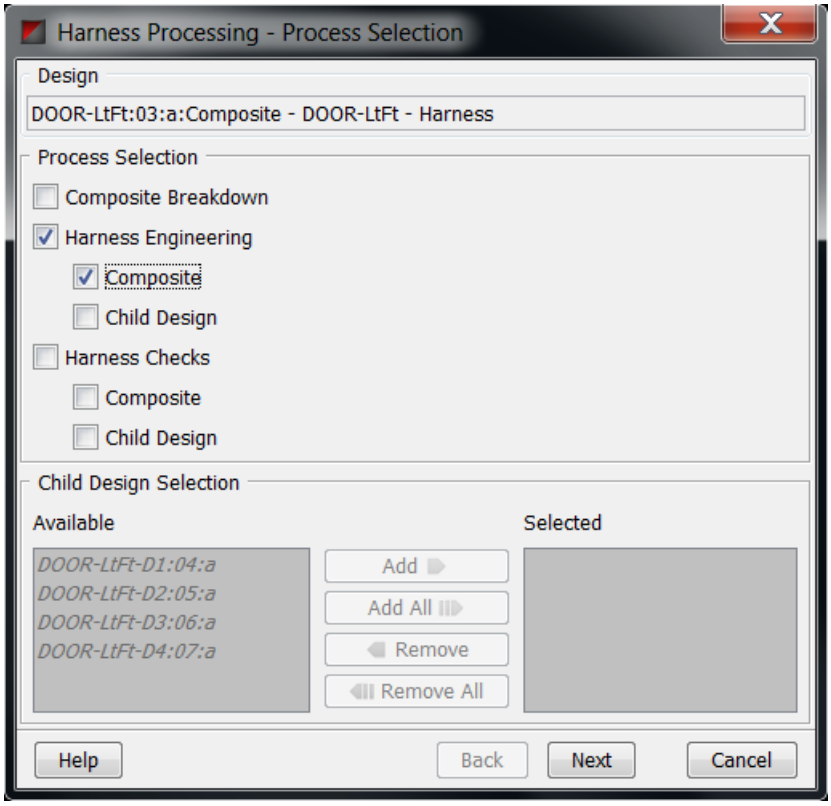
The change in version number confirms that your new plugin has been loaded.

i. Close the Plugins dialog.

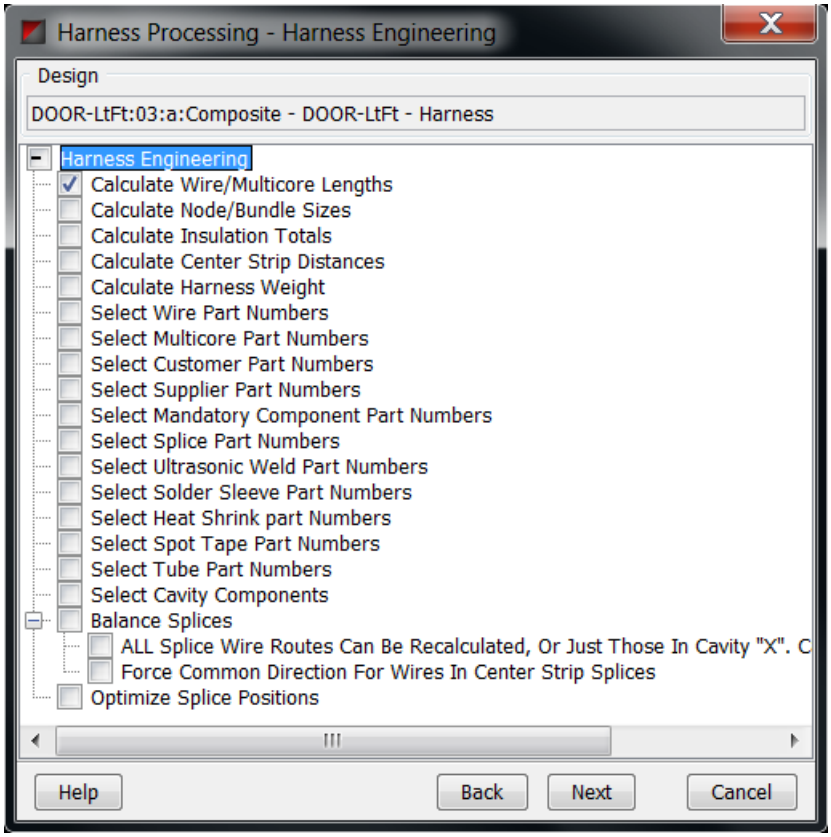
4) At this point, you are in a position to try your new plugin.

a. Open your diagram

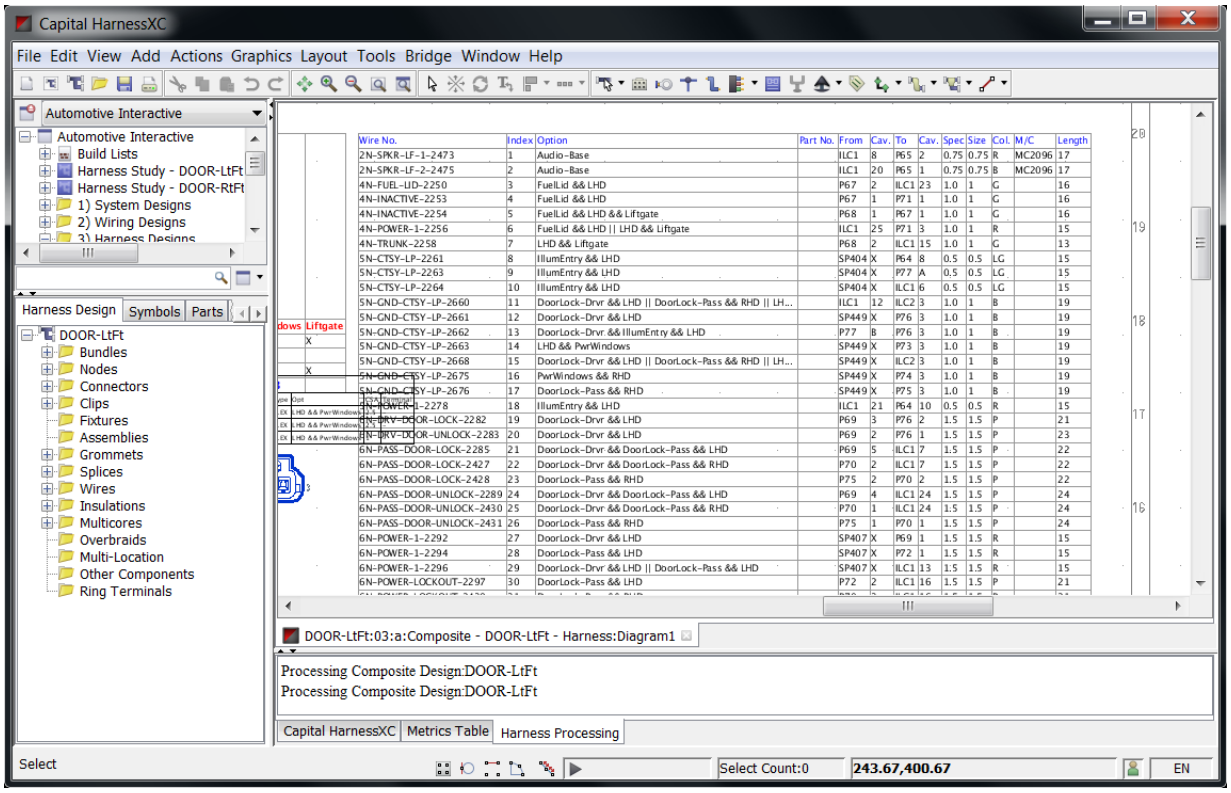
b. On the main menu of Capital HarnessXC, click the Tools > Harness Processing item. This will open the following window:



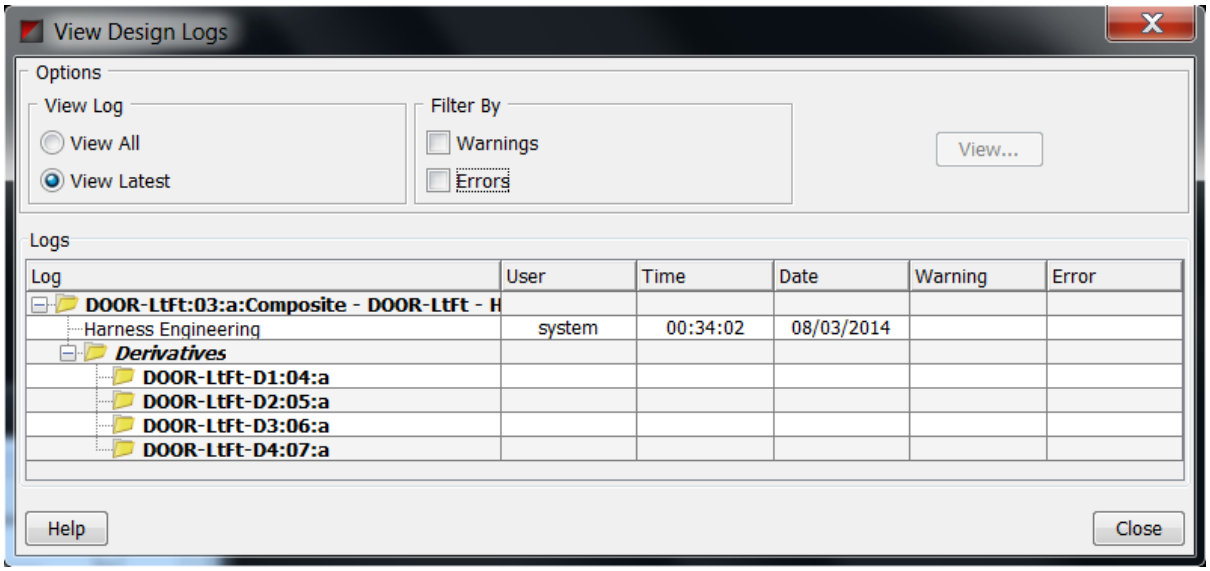
c. Select the “Harness Engineering” option on the “Composite” as in the screenshot above, then press “Next”:



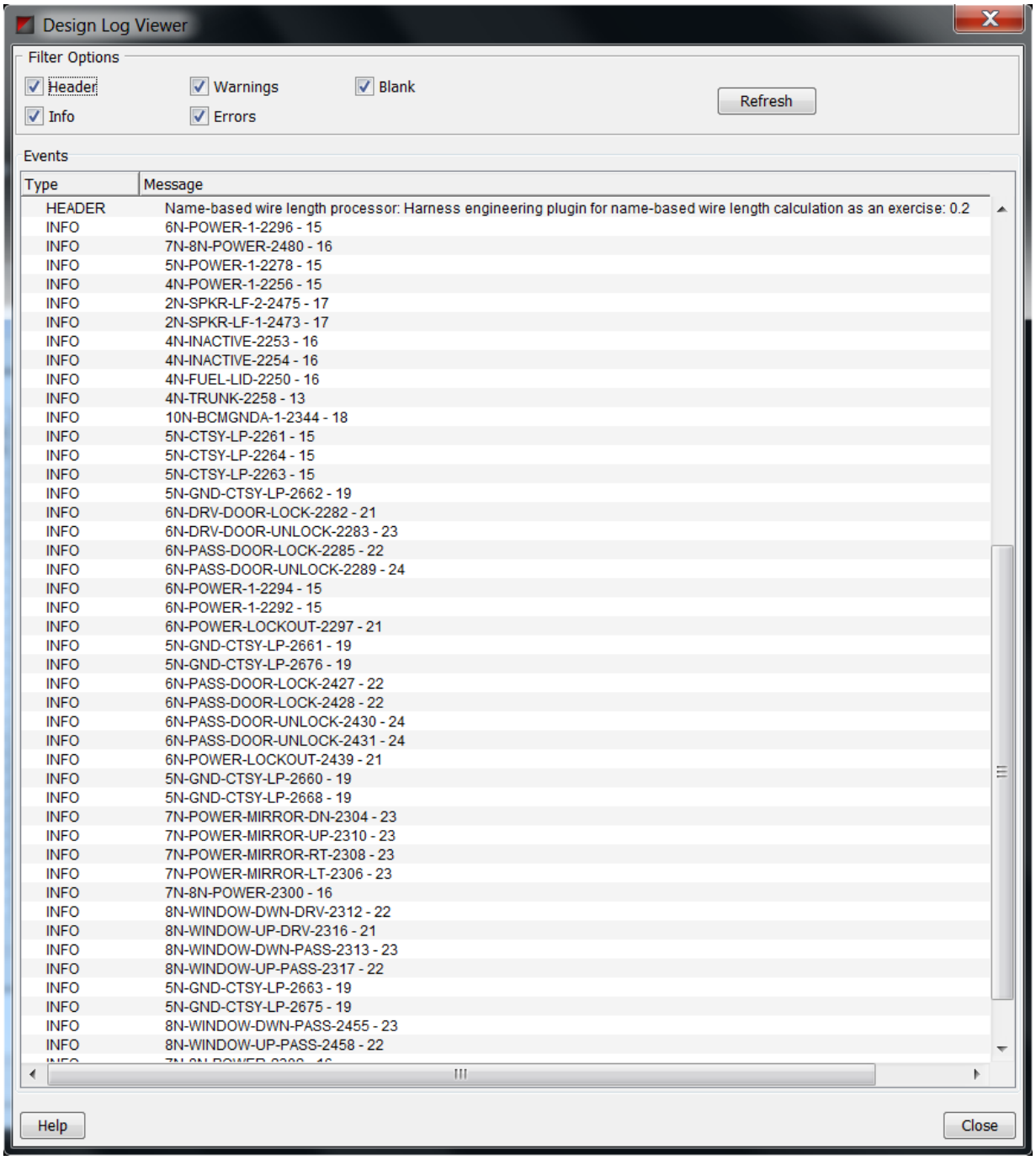
d. Select only the first “Calculate Wire/Multicore Lengths” option as in the screenshot above, the press “Next” and “Finish”: the processing will then take place (this is when your plugin will be invoked) and the diagram will be re-opened on completion. You can then zoom to the Wire Table to see the Length values have changed and now correspond to the calculation you have provided in the plugin (i.e. the wire name length):



e. On the main menu of Capital HarnessXC, click the Tools -> Design Log -> View item. This will open the following window:



f. Make sure you de-select the “Filter By > Warnings, Errors” options as in the screenshot above. Double click the “Harness Engineering” row, this will open the log content window:



Notice the first “HEADER” message shown in the log table in the screenshot above is added by the processing engine just before executing a plugin. The following “INFO” messages are the ones which have been added by the plugin itself.

At this point, you should know the following:

- How to iteratively develop Java Custom harness processors.

Guidance

Always develop custom harness processors in Java for performance. Also ensure that any debug messages are removed or disabled before your plugin is deployed.

Examples

At this point, you should have a good idea how to implement this type of plugin. Some examples have been provided to help you. These examples can be found in the following location within the Capital installation:

```
doc/plugin/examples/Java/src/com/example/plugin/harnessprocessing
```

You can copy and modify the examples to implement your own plugins if you wish. However, you need to check the license and copyright information at the start of the files before you do this.

The examples provided may be different in future releases of Capital. If you do use these examples, therefore, you need make a copy of them (we reserve the right to add, remove or modify these examples).